Mid-Term Lab Examination — 8-bit Accumulator

University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Mid-Term Lab Examination | Group C1 | Spring 2026

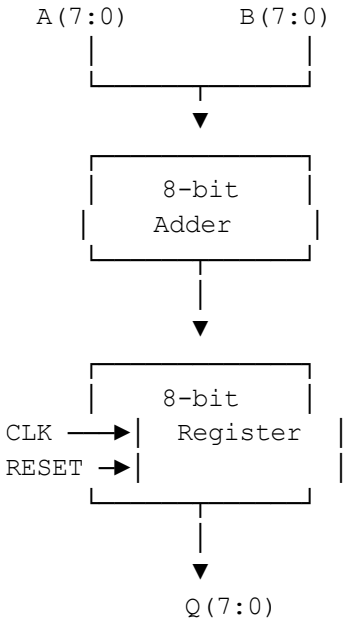
Duration: 1 hr Total Marks: 20

Student Name: Md. Akif Hossain ID: 22201029

Mid-Term Task: 8-bit Accumulator

Objective. Design and implement an 8-bit accumulator in VHDL by combining the 8-bit adder and register concepts developed in the laboratory. The system must perform addition synchronously with the clock and store the result in an 8-bit register.

System Architecture



Functional requirement.

At every rising edge of CLK, the register stores the sum of the two 8-bit inputs. RESET has priority over the addition operation.

Required Operation

RESET	CLK	A, B	Operation / Q
1	↑	XX, XX	Q ← 00000000
0	↑	A, B	Q ← A + B
0	No rising edge	XX, XX	Q remains unchanged

Required Test Sequence

Your testbench must produce the following sequence. The expected Q value should be observed after the corresponding rising clock edge.

Step	RESET	A	B	Operation	Expected Q / Explanation
1	1	00000000	00000000	Reset	00000000 — Register is cleared
2	0	00000011	00000101	3 + 5	00001000 — Load sum
3	0	00000010	00000001	2 + 1	00000011 — Load new sum
4	0	11111111	00000001	255 + 1	00000000* — 8-bit overflow
5	0	10101010	00000101	170 + 5	10101111 — Load new sum

* Note: For Step 4, the mathematical result is 256, but an 8-bit output can represent only 0–255. Therefore, Q = 00000000 and the carry-out is discarded unless your implementation explicitly exposes it.

Implementation Requirements

• Use VHDL and IEEE.STD_LOGIC_1164.ALL.

• The register operation must be synchronized with the rising edge of CLK.

• RESET must be synchronous and active-high.

• Do not use a behavioral integer-based accumulator as a replacement for the required adder/register structure.

• The design should be modular and clearly identify the 8-bit adder and 8-bit register portions.

• The submitted testbench must generate a clock and apply the required input sequence.

Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

• Attach your entity code and testbench code

• Attach your output visuals

• Challenges faced during the experiment

• Conclusion

accumulator_8bit.v

```
`timescale 1ns/1ps
module accumulator_8bit (
    input  [7:0] A,
    input  [7:0] B,
    input        CLK,
    input        RESET,
    output reg [7:0] Q
);
always @(posedge CLK) begin
    if (RESET)
        Q <= 8'b00000000;
    else
        Q <= A + B;
end
endmodule
```

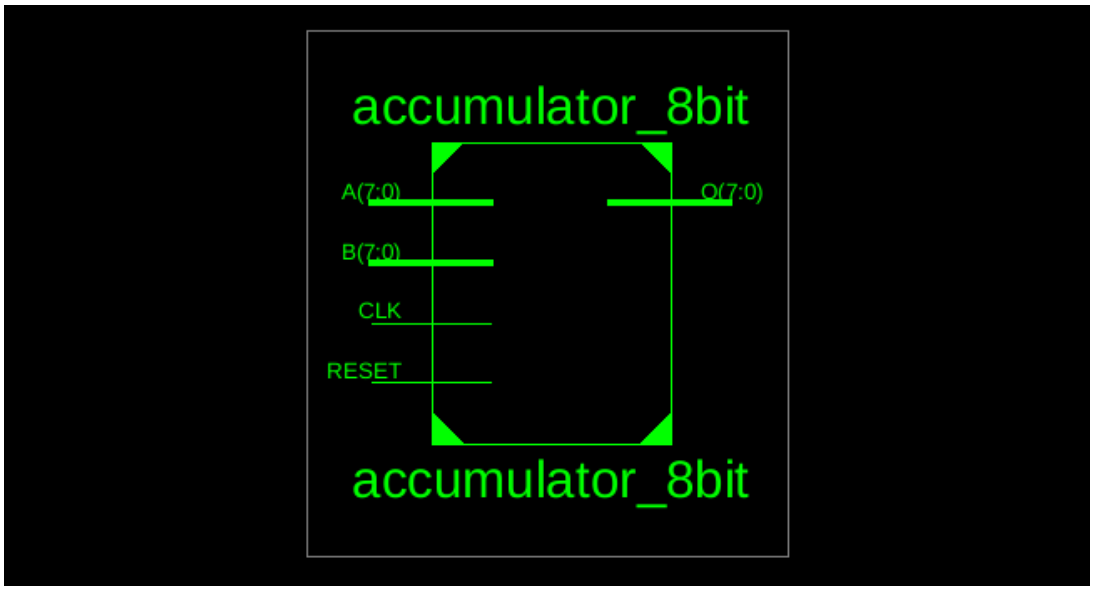


Fig: Accumulator 8 bit RTL schema

accumulator_8bit_tb.v

```
`timescale 1ns/1ps
module accumulator_8bit_tb;
reg [7:0] A, B;
reg CLK, RESET;
wire [7:0] Q;
accumulator_8bit uut (
    .A(A),
    .B(B),
    .CLK(CLK),
    .RESET(RESET),
    .Q(Q)
);
always #5 CLK = ~CLK;
initial begin
    CLK = 0;
    RESET = 0;
    A = 0;
    B = 0;
    //  Reset
    RESET = 1;
    #10;
    // 3 + 5 = 8
    RESET = 0;
    A = 8'b00000011;
    B = 8'b00000101;
    #10;
    // 2 + 1 = 3
```

```
    A = 8'b00000010;
    B = 8'b00000001;
    #10;
    // 255 + 1 = 0 (overflow)
    A = 8'b11111111;
    B = 8'b00000001;
    #10;
    // 170 + 5 = 175
    A = 8'b10101010;
    B = 8'b00000101;
    #10;
    $finish;
end
endmodule
```

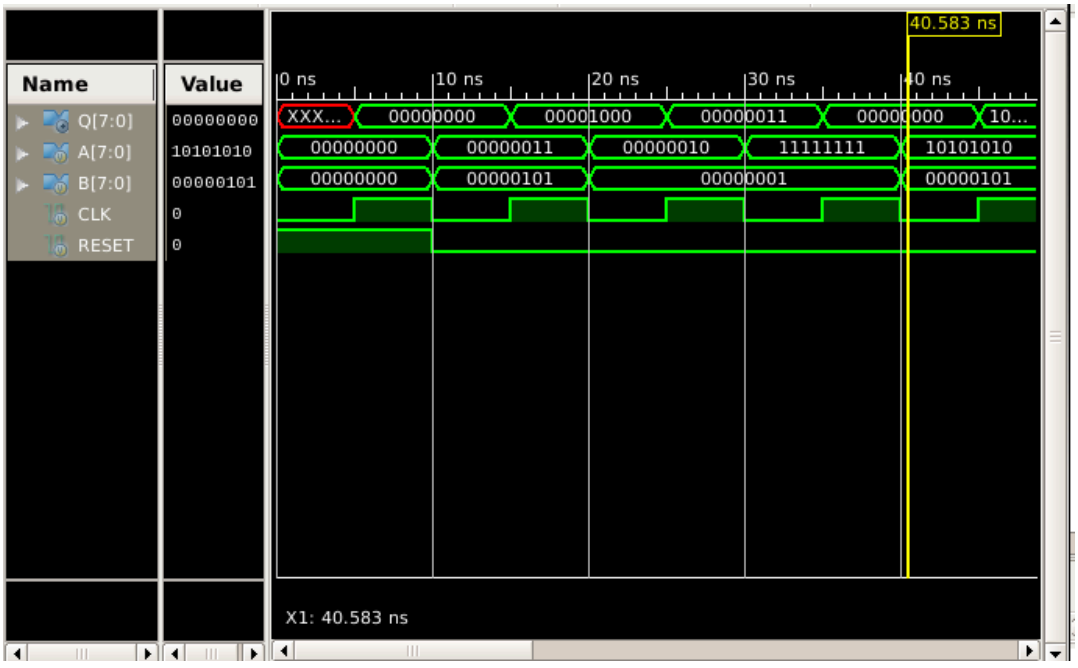


Fig: **Accumulator 8 bit Waveform**

Expected Q

Step	A	B	Q
1	00000000	00000000	00000000
2	00000011	00000101	00001000
3	00000010	00000001	00000011
4	11111111	00000001	00000000
5	10101010	00000101	10101111

Github Repositories:

https://github.com/Akifhossain29/ISE-14.7-VLSI

Challenges faced during the experiment

The major problems that arose during the experiment were designing the correct 8 bit adder and the register, and ensuring that the register worked in sync with the rising edge of the clock. The active high reset was also a tricky issue to handle and the 8 bit overflow condition was also not easy to understand. The input sequence and timing of the clock had to be kept properly during simulation to get the required output values.

Conclusion

Design and simulation of 8-bit accumulator was successfully done in the verilog language. The accumulator was able to add the two 8-bit inputs and store the result in the register on each positive edge of the clock. The synchronous reset successfully reset the register to 00000000 and the overflow case 255 + 1 was found to be 00000000 as the carry-out was ignored. So, through the test bench the need for the functionality of the 8-bit accumulator was verified.